AWS VPC PEERING

Student Project Report

Build • Test • Explain

Student Name	**Sarthak Sharma**
Batch / Class	**EWB Internship Batch 1**
Project	AWS VPC Peering — private connectivity between two VPCs
AWS Region	Mumbai (ap-south-1), as shown in the AWS console
Completion date	30 October 2026

Project objective

Create a private connection between two non-overlapping Amazon VPCs using VPC peering, route tables and security groups. Keep the application instances private and use a public bastion host for SSH administration.

Report status: main peering and forward connectivity test verified. Items not yet evidenced are clearly marked as pending rather than assumed complete.

1. Executive Summary

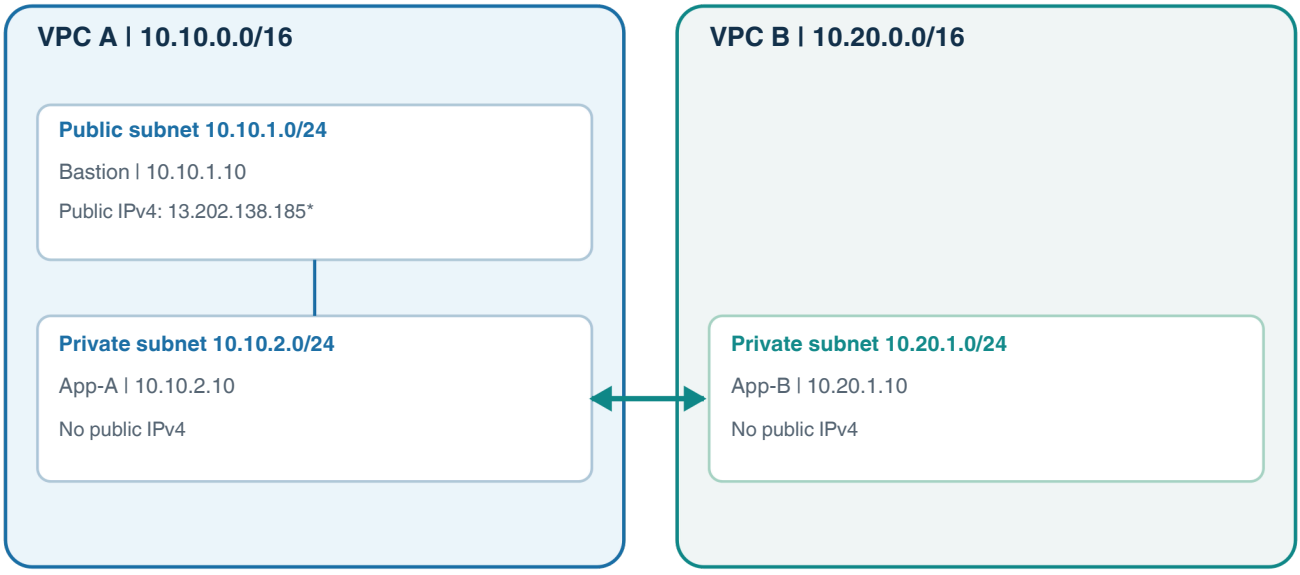
Thelab implemented a two-VPCAWS network using private IPv4 addressing. VPC A hosts a public bastion and private App-A; VPC B hosts private App-B. A VPC peering connection was created and activated, and routes were added to permit private traffic between the VPC CIDR ranges.

The following results were directly observed in the provided terminal screenshots:

Test / checkpoint	Observed result	Status
SSH to bastion	Amazon Linux 2023 shell reached as ec2-user; host private IP shown as 10.10.1.10.	Verified
SSH to App-A via ProxyJump	Amazon Linux 2023 shell reached; prompt showed 10.10.2.10.	Verified
Baseline App-A → App-B ping	4 packets transmitted, 0 received, 100% packet loss before peering/routes.	Verified
Peering connection	Connection ewb-a-b-peer; ID pcx-06e43bbc6454d9363; status Active.	Verified
Post-peering App-A → App-B ping	4 packets transmitted, 4 received, 0% packet loss; replies from 10.20.1.10.	Verified
Reverse ping App-B → App-A	No screenshot/result supplied in the conversation.	Verified
SSH to App-B / controlled failure / cleanup	Not confirmed in the evidence provided.	Verified

Conclusion: the active peering connection and successful forward ping demonstrate private connectivity from App-A to App-B after the route changes. The full workbook submission asks for additional evidence, includingreverseping, App-B SSH, controlled failure/recoveryand cleanup.

2. Architecture and Address Plan



*Public IP recorded during this lab; verify current address before reuse.

The two VPC CIDRs do not overlap. App-A and App-B have private addresses only; the bastion is the administration entry point. VPC peering provides a private route between VPCs but does not automatically create routes or provide VPC B with internet access.

3. Resource Inventory

Resource	Name / ID	CIDR / address / role
VPC A	ewb-vpc-a vpc-0f6d914ec6c8a08a5	10.10.0.0/16
VPC B	ewb-vpc-b VPC ID not recorded in available evidence	10.20.0.0/16
Public subnet A	ewb-a-public	10.10.1.0/24
Private subnet A	ewb-a-private	10.10.2.0/24
Private subnet B	ewb-b-private	10.20.1.0/24
Bastion EC2	ewb-bastion	10.10.1.10; public IPv4 observed as 13.202.138.185 during connection
App-A EC2	ewb-app-a	10.10.2.10; no public IPv4 intended
App-B EC2	ewb-app-b	10.20.1.10; no public IPv4 intended
Peering	ewb-a-b-peer pcx-06e43bbc6454d9363	Active; same AWS account and Region
Key pair	ewb-lab	Private key stored locally at %USERPROFILE%\.ssh\ewb-lab.pem; do not include key material in submission.

Note: route table and security group IDs were not captured in the evidence available for this report. The resource names and expected settingsbelowfollow theprojectworkbookand the implementation stepsdiscussed.

4. Routing and Security Design

Required route table entries after peering

Route table	Destination	Target	Purpose
ewb-rt-a-public	10.10.0.0/16	ewb-igw-a	Internet access for public bastion; retain local route.
ewb-rt-a-public	10.20.0.0/16	pcx-06e43bbc6454d9363	Allows bastion-side routing to App-B through peering.
ewb-rt-a-private	10.20.0.0/16	pcx-06e43bbc6454d9363	Forward route from App-A to VPC B.
ewb-rt-b-private	10.10.0.0/16	pcx-06e43bbc6454d9363	Return route from App-B to VPC A.

Security group intent

Security group	Inbound rule expected	Security rationale
ewb-sg-bastion	SSH TCP 22 from current client public IP /32	Restricts administration to the student's current public IP.
ewb-sg-app-a	SSH TCP 22 from 10.10.1.10/32; ICMP echo from 10.20.1.10/32	Allows SSH from bastion and ping replies/requests from App-B as defined by lab.
ewb-sg-app-b	SSH TCP 22 from 10.10.1.10/32; ICMP echo from 10.10.2.10/32	Allows administration through bastion and ping from App-A only.

Default outbound allow-all rules were assumed to remain unchanged, as in the lab design. Do not expose SSH or ICMP to 0.0.0.0/0. Confirm actual saved rules in the console before final submission.

5. Implementation Summary

1. Created VPC A(10.10.0.0/16)and VPC B (10.20.0.0/16) with non-overlapping CIDR ranges.

2. Created the public subnet in VPC A and private subnets in both VPCs; configured the Internet Gateway and route table associations for the bastion network.

3. Launched three Amazon Linux 2023 instances: bastion (10.10.1.10), App-A (10.10.2.10), and App-B (10.20.1.10). App-A and App-B were designed to have no public IPv4 addresses.

4. Configured Windows OpenSSH using the local ewb-lab.pem key and SSH ProxyJump aliases. Successfully reached the bastion, then App-A through the bastion.

5. Ran the baseline ping from App-A to App-B before peering. It returned 100% packet loss, as expected when no peering path/routes were configured.

6. Created and accepted the VPC peering connection ewb-a-b-peer. The AWS console displayed status Active and peering ID pcx-06e43bbc6454d9363.

7. Added the three required peering routes to the A-public, A-private and B-private route tables. The subsequent

Evidence item	Output / observation	Interpretation
Bastion SSH login	Amazon Linux 2023 prompt; private host 10.10.1.10	Public entry point reachable over SSH.
App-A SSH login	Amazon Linux 2023 prompt; private host 10.10.2.10	ProxyJump to private App-A works.
Baseline ping	4 transmitted; 0 received; 100% loss	No working inter-VPC path before peering/routes.
Peering status	ewb-a-b-peer — Active	Peering link established; routes still required.
Forward ping after routes	4 transmitted; 4 received; 0% loss; replies from 10.20.1.10	App-A can reach App-B over private IPs.

Recommended screenshots to insert before submission

- AWS route tables showing all three peering routes and subnet associations.
- App-A successful ping screenshot (already provided in the conversation; attach or paste it into the final submitted copy).
- Reverse ping from App-B to App-A (not yet evidenced).
- SSH session to App-B from Windows using `ssh ewb-app-b` (not yet evidenced).
- Controlled ICMP failure after temporarily removing App-B's inbound ICMP rule, followed by restored successful ping (not yet evidenced; skipped by request).
- Cleanup evidence showing lab resources terminated/deleted (not yet evidenced).

7. Traffic Flow Explanation

App-A to App-B: App-A sends ICMP traffic to 10.20.1.10. The route table associated with App-A's subnet sends destination 10.20.0.0/16 to the VPC peering connection. VPC B's route table sends return traffic for 10.10.0.0/16 back through the same peering connection. The App-B security group must allow the ICMP request from App-A's private IP. Laptop to App-B administration: the laptop initiates SSH to the bastion's public IPv4 address. SSH ProxyJump then opens a separate SSH connection from the bastion to App-B's private address 10.20.1.10. This requires the VPC A public route table to route 10.20.0.0/16 to the peering connection, the VPC B return route to VPC A, and security group rules that permit SSH from the bastion's private IP.

VPC peering is not transitive routing and does not grant VPC B general internet access through VPC A. Each VPC needs appropriateroutes andsecurityrules.

8. Limitations and Remaining Work

This report is based on theworkbook and thescreenshots/terminaloutput shared during implementation. It records only results directly observed. The reverse-direction ping, App-B SSH login, controlled ICMP failure/recovery, and cleanup have not been confirmed in the evidence supplied. The workbook lists these as submission evidence, so they should be completed or explicitly discussed with the instructor before presenting this as a fully complete submission. The controlled failure test was skipped at the student's request. No claim is made that this test passed. No screenshot ofcleanupor billing verification wasprovided.

9. Cleanup and Cost Safety

Aftersaving all evidence, terminatethe three lab EC2instances and verify that their lab EBS volumes are deleted. Remove any lab snapshots or Elastic IPs created. Delete the peering connection and peer routes, then remove lab security groups, subnets, custom route tables, the Internet Gateway, and both VPCs in dependency order. Remove the lab key pair if no longer needed. Finally, check the AWS Billing console for remaining chargeable resources; billing information may take time to update.

Do not terminate or delete resources until you have collected the screenshots required for submission. EC2, EBSstorageand publicIPv4usage may incur charges.

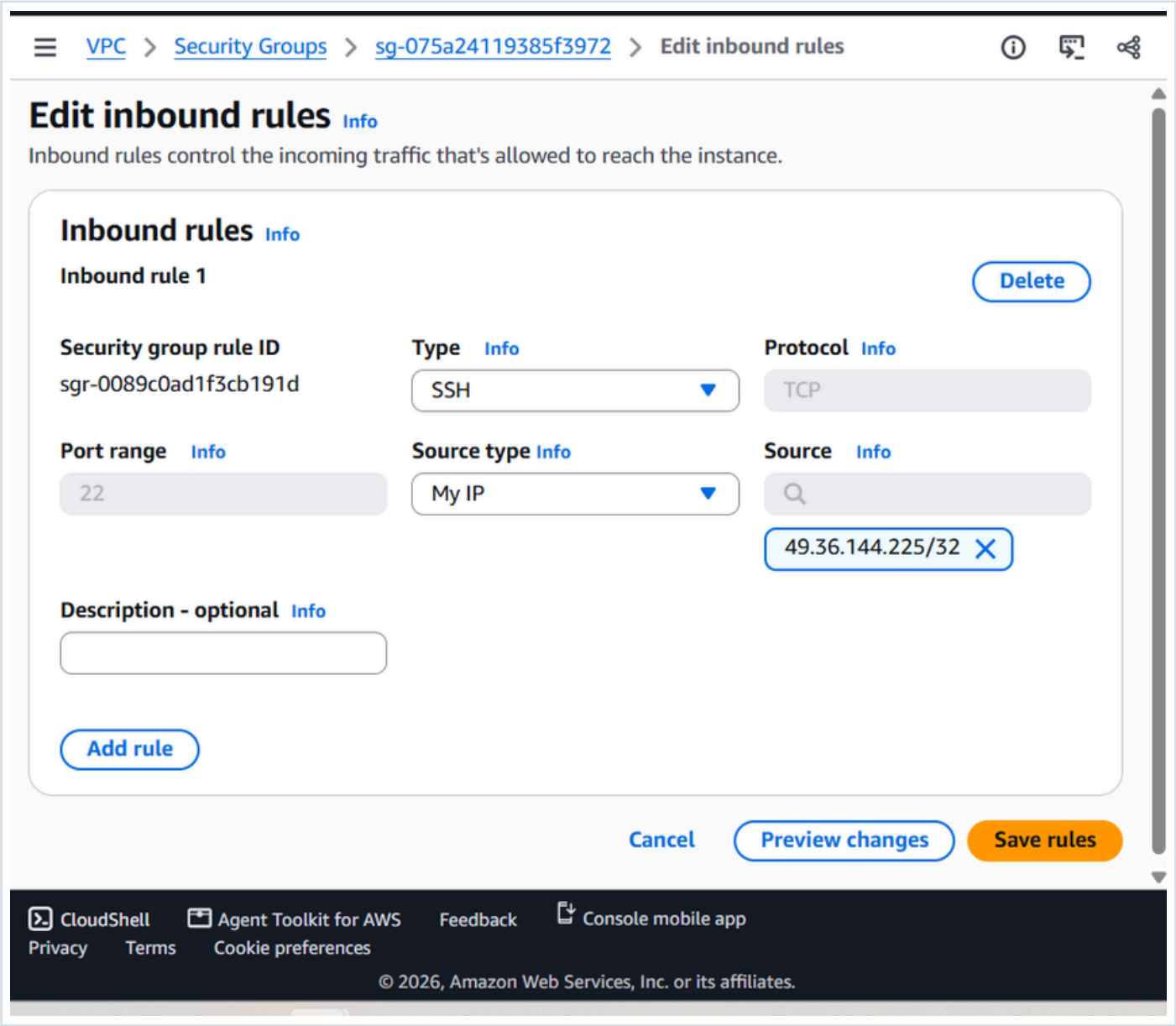
10. Conclusion

Theproject demonstratedhow VPC peering, route tables and security groups work together to provide private connectivity between separate AWS VPCs. The baseline failure followed by successful App-A-to-App-B ping after peering and route configuration is the central verified result. Additional bidirectional, SSH, controlled-failure and cleanup evidence remains to be added for a complete workbook-aligned submission.

Submission filename suggested by workbook: YourName_EWB_VPC_Peering.pdf. Replace the student name and batch placeholders and attach the original screenshots before submitting.

Evidence 1 — Bastion security group SSH rule

InboundSSHis configured forTCPport22fromthestudent'spublicIPv4address/32.Verify this remains the current public IP before final submission.



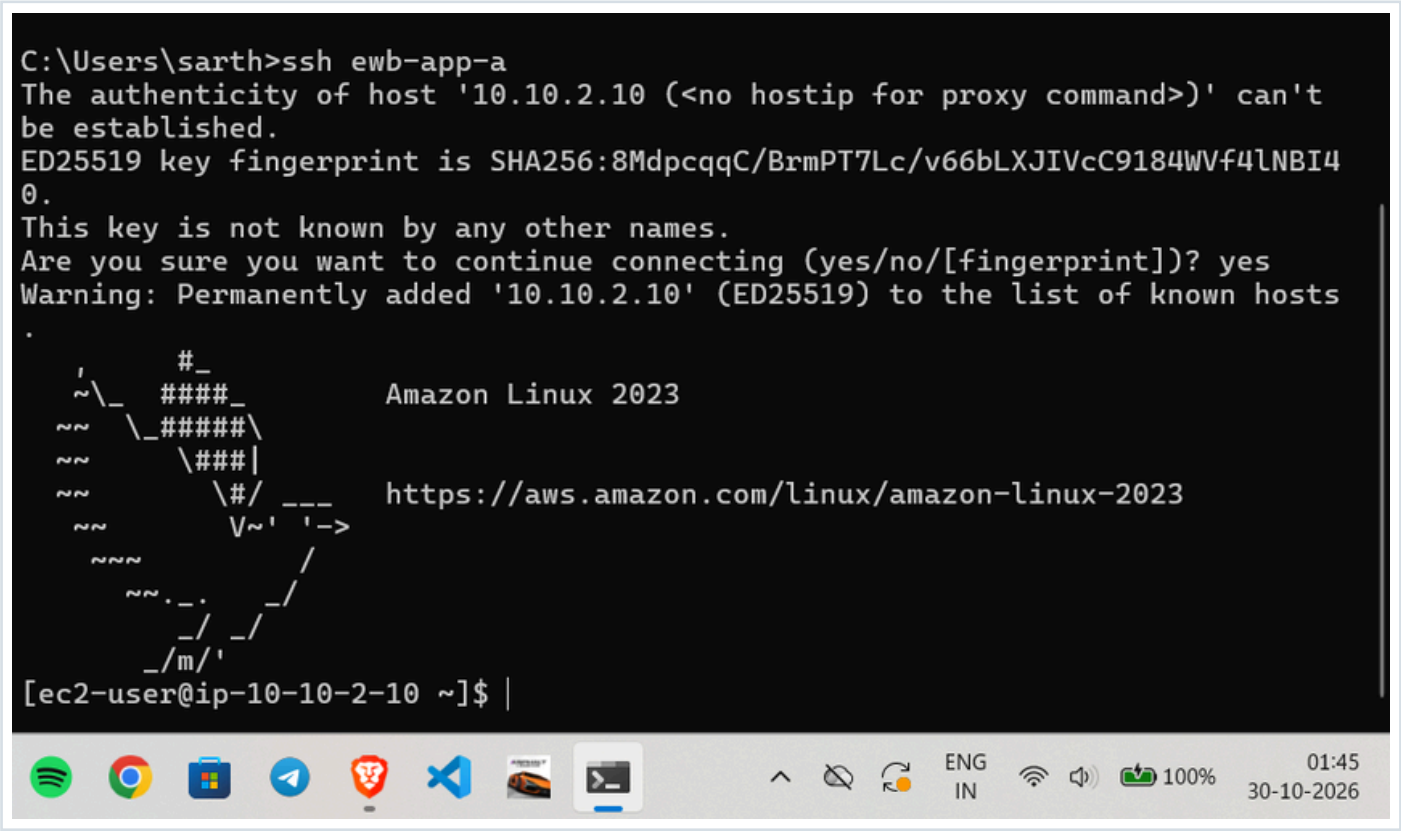
Evidence 2 — Successful SSH login to the bastion

The terminal displays the Amazon Linux 2023 shell as ec2-user on the bastion host (private address 10.10.1.10).



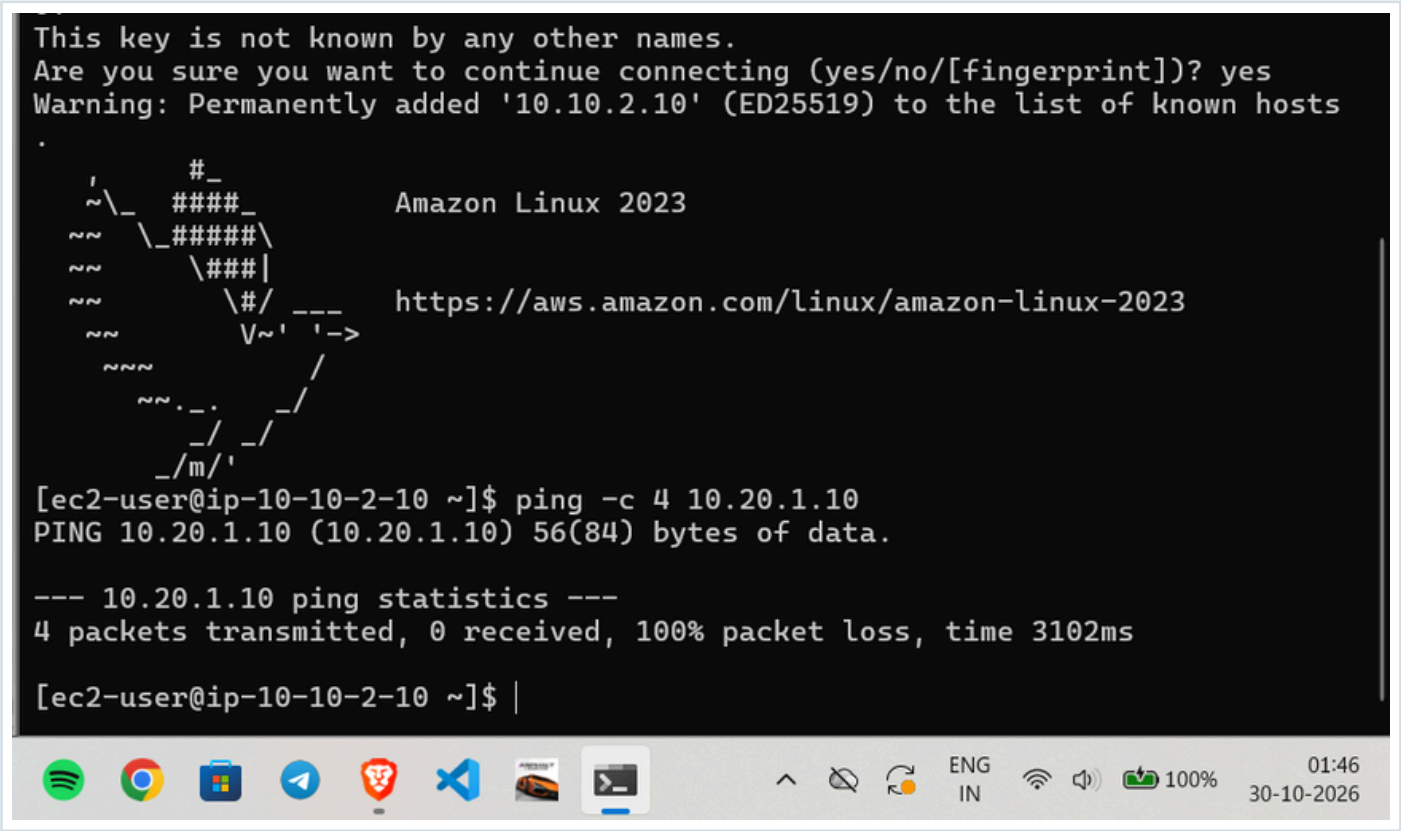
Evidence 3 — Successful SSH login to App-A

TheterminalpromptidentifiesApp-Aat10.10.2.10,reachedthroughtheWindowsSSH ProxyJump configuration.



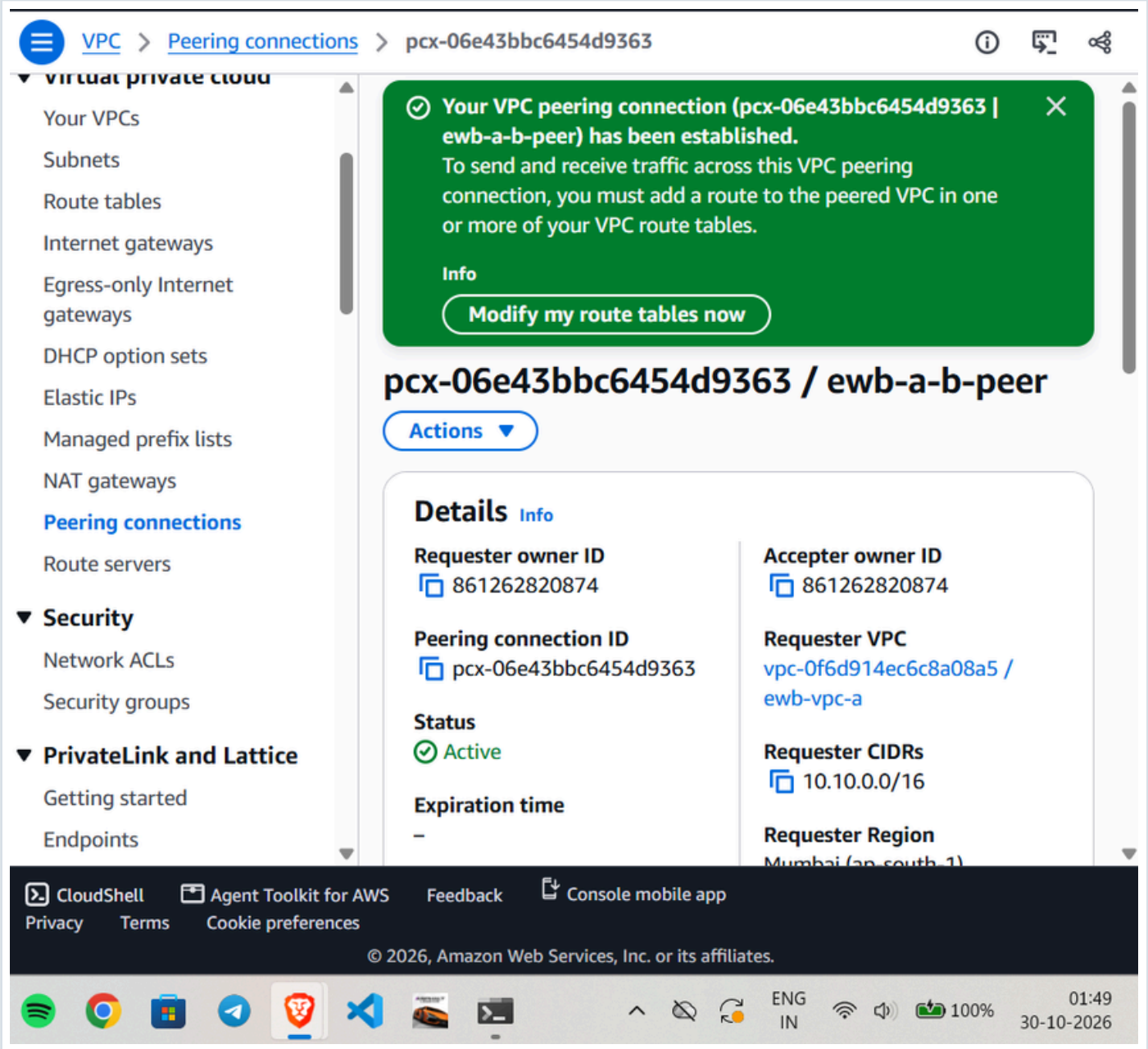
Evidence 4 — Baseline ping before VPC peering

App-A sent fourpingpackets toApp-Bat10.20.1.10;zerowerereceived(100%packetloss), as expected before the peering route was established.



Evidence 5 — VPC peering connection is Active

TheAWSconsoleshowsewb-a-b-peerwithpeeringIDpcx-06e43bbc6454d9363andstatus Active.



Evidence 6 — Successful ping after peering and routes

App-A received all four replies from App-B at 10.20.1.10 (0% packet loss), confirming forward private connectivity.

